Pràctica 3: Part 1

1 Introducció

En aquesta pràctica es proposa el desenvolupament d'una aplicació per a la gestió d'una biblioteca mitjançant programació orientada a objectes. L'objectiu és modelar adequadament les entitats del sistema i les relacions entre elles, aplicant els principis d'encapsulació, herència i polimorfisme. S'espera que l'alumnat realitzi un disseny coherent del model de classes abans de procedir a la implementació, diferenciant clarament les responsabilitats de cada classe i evitant duplicacions innecessàries de codi.

A diferència de les pràctiques anteriors, la Pràctica 3 es desenvoluparà en dues parts. A la part 1 de la pràctica, s'implementarà una aplicació seguint el paradigma de programació orientada a objectes que modeli el sistema de gestió de prestecs. En aquesta aplicació, la vista serà implementada com una interfície de text.

A la part 2 de la pràctica, es proposa reemplaçar la vista implementada a la part 1 per una interfície gràfica, aplicant conceptes de programació orientada a esdeveniments.

Aquest document descriu la part 1 del treball a realitzar.

2 Descripció del Problema

Es vol desenvolupar un sistema que permeti gestionar els usuaris d'una biblioteca, els exemplars disponibles i els préstecs realitzats. Existeixen diferents tipus d'usuaris i diferents tipus de préstecs, el comportament dels quals dependrà de la seva naturalesa concreta.

Un usuari, emmagatzema el seu correu electrònic, nom i adreça. Existeixen dos tipus d'usuaris: estudiant i professor. Cada tipus disposa d'un nombre màxim de préstecs normals i d'un nombre màxim de préstecs llargs que pot tenir actius simultàniament. L'estudiant pot tenir 2 prestes normals i 1 prestec de llarg termini, mentres que el professor pot tenir 2 prestecs normals i 2 prestec llargs.

Un exemplar correspon a una còpia física d'un llibre i disposa d'un identificador numèric, autor i títol. A més, emmagatzema un indicador que assenyala si permet préstecs de llarg termini i un altre que indica si es troba disponible. Un exemplar únicament podrà prestar-se quan estigui disponible i només podrà generar un préstec llarg si la seva configuració ho permet.

Un préstec emmagatzema la data de creació, la data límit de devolució i un indicador que assenyala si ha estat retornat, a més de mantenir la referència a l'usuari que realitza el préstec i a l'exemplar prestat. Existeixen dos tipus de préstec: el prestec normal i el prestec de llarg termini. El prestec normal té una durada de 7 dies, que per facilitar el desenvolupament de la pràctica, serà de 70 segons. D'altra banda, el preste de llarg termini té una durada de 14 dies, o de 140 segons dins d'aquesta pràctica. En el moment de crear un préstec, l'exemplar passa automàticament a no estar disponible i s'estableix la data de creació del prestec i també la seva data límit corresponent segons el tipus de préstec. Quan un préstec es retorna, es marca com a retornat i l'exemplar torna a estar disponible.

Si un usuari de la biblioteca té algun prestec endarrerit (es a dir, excedeix la data límit de retorn), no podrà obtenir prestecs nous fins a que es produeixi el retorn dels exemplars amb la data límit exhaurida.

Pots consultar la Figura 1 per tenir una visió general de l'evolució d'un préstec.

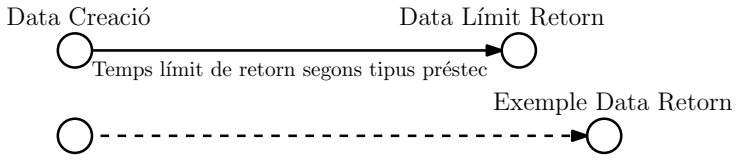

Figura 1: Evolució temporal d'un préstec

La gestió del sistema es realitzarà mitjançant llistes d'usuaris, exemplars i prestecs.

3 Objectiu i Funcionalitats

L'objectiu de l'aplicació és permetre la gestió bàsica del funcionament de la biblioteca garantint el compliment de les restriccions establertes per a usuaris, exemplars i préstecs.

L'aplicació haurà de permetre, com a mínim, les operacions següents:

1. **Gestió d'exemplars**:
   (a) **Afegir Exemplar**: Donar d'alta un exemplar indicant les seves dades i si permet préstec llarg.
   (b) **Visualitzar Exemplars**: Mostrar la llista completa d'exemplars.
2. **Gestió d'usuaris**:
   (a) **Afegir Usuari**: Donar d'alta un usuari indicant el seu tipus (estudiant o professor).
   (b) **Visualitzar Usuaris**: Mostrar la llista completa d'usuaris.
3. **Gestió de préstecs**:
   (a) **Afegir Prestec**: Crear un préstec per a un usuari i exemplar. **Per identificar l'usuari i exemplar involucrats, farem servir els seus índexs dins de les llistes (ArrayLists) que utilitzarem per emmagatzemar-los**.
   (b) **Retornar Prestec**: Retornar un préstec.
   (c) **Visualitzar Prestecs**: Mostrar la llista de tots els préstecs.
   (d) **Visualitzar Prestecs no Retornats**: Mostrar la llista de préstecs no retornats.
4. **Guardar Dades**: Guardar les dades de l'aplicació.
5. **Carrega Dades**: Carregar les dades de l'aplicació.

6. **Sortir**: Surt de l'aplicació.

El sistema haurà de comprovar que l'exemplar es trobi disponible abans de realitzar un préstec, que l'usuari no tingui prestecs endarrerits, que no superi el seu nombre màxim de préstecs normals o llargs actius i que no es pugui realitzar un préstec llarg si l'exemplar no ho permet. La implementació haurà d'aplicar correctament els principis de la programació orientada a objectes, fent un ús adequat de l'herència i el polimorfisme.

4 Material per a la Pràctica

Per a aquesta pràctica es proporcionen les següents classes i interfícies:

- **Menu**: classe útil per implementar els menús textuals descrits anteriorment.
- **InDades**: interfície que ha de ser implementada per la classe `Dades`.
- **InExemplar**: interfície per a la classe `Exemplar`.
- **InUsuari**: interfície que ha d'implementar la classe `Usuari-`
- **InPrestec**: interfície disenyada per a la classe `Prestec`.
- **Llista**: classe genèrica per implementar llistes que necessitarem en el nostre sistema d'informació. El fitxer conté l'estructura general de la classe, incloenthi atributs, però només els prototips dels mètodes, quedant pendent la definició del cos d'aquests.
- **InLlista**: interfície que ha de ser implementada per la classe `Llista`.

5 Descripció de la Pràctica

A continuació, es detallen una sèrie de passos per a resoldre la pràctica que s'ha proposat. Es recomana seguir aquests passos.

5.1 Creació del projecte

El primer pas serà crear un projecte IntelliJ, el qual s'ha d'anomenar `Cognom1Nom1Cognom2Nom2_P31`, on 1 i 2 fan referència als dos membres de la parella, tenint en compte les consideracions següents:

- La primera lletra de cada part en majúscula i la resta en minúscula.
- S'han d'evitar els accents i caràcters especials com ñ o ç.

Per exemple, una estudiant amb nom Dolça Martínez Castaña, hauria de crear un projecte amb el nom `MartinezCastanaDolca`.

- La classe principal s'ha de denominar `IniciadorBiblioUB`, i el paquet per defecte `prog2.vista`. En el nom del paquet s'han d'utilitzar també els mateixos criteris anteriors.
- A IntelliJ s'ha d'indicar que la classe principal sigui `prog2.vista.IniciadorBiblioUB`.

5.2 Paquets del projecte

El projecte que hem de desenvolupar està basat en el patró Model-Vista-Adaptador. Haurem de crear una classe `Adaptador`, de manera que la vista no accedirà directament a les dades del model, sinó que ho farà a través d'aquesta classe. En conseqüència, el projecte estarà format per 3 paquets:

- **prog2.vista:** la vista contindrà totes les classes relacionades amb el maneig de menú d'opcions. Entre elles, destaca la classe `BiblioUB` donat que implementarà el menú d'opcions ofert per l'aplicació.
- **prog2.adaptador:** el paquet adaptador únicament contindrà la classe `Adaptador` , que funcionarà com a mediador entre la vista i el model. La vista només podrà utilitzar aquesta classe per accedir a la informació del model.
- **prog2.model:** el paquet model contindrà totes les classes que modelen les dades que s'han de gestionar dins de l'aplicació. Dins el model, mereix especial menció la classe `Dades`, que contindrà totes les dades de l'aplicació i portarà a terme totes les accions que afecten els mateixos.

A la Figura 2 es mostren les relacions entre les classes principals de cadascun dels paquets.

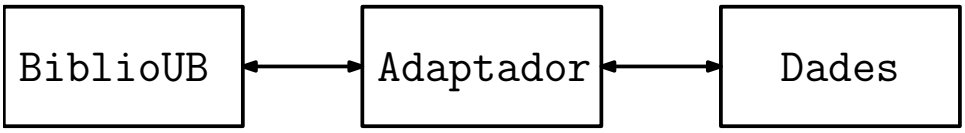

Figura 2: Principals classes del patró Model-Vista-Adaptador i les seves relacions

5.3 Classes del Projecte

A continuació es descriuen les principals classes que s'han d'implementar dins de la nostra aplicació.

Classe Exemplar

La classe `Exemplar` emmagatzema tota la informació rellevant de cada exemplar. S'ha d'afegir un constructor que permeti inicialitzar els valors dels atributs de la classe (per a més informació sobre aquests atributs, es pot consultar la secció 2 de descripció del problema). A més, s'han d'afegir els corresponents setters i getters.

S'ha de crear també un mètode `toString` que permeti mostrar la informació de l'exemplar. La cadena de text generada hauria de ser semblant a la següent:

```
Id=0, Titol=El Quijote, Autor=Miguel de Cervantes Saavedra, Admet
Prestec Llarg=true, Disponible=true
```

El codi base incorpora la interfície `InExemplar`, que la classe `Exemplar` ha d'implementar.

Classe Usuari i Classes Filles

La classe `Usuari` guardarà la informació dels usuaris. Aquesta informació inclou el correu electrònic, nom i adreça de cadascun. S'ha de declarar la classe `Usuari` com a classe abstracta, incorporant un constructor que inicialitzi els seus atributs, així com els getters i setters necessaris.

A més, també necessitem definir un mètode `toString` que ha de crear una cadena de text semblant a la següent:

```
Tipus=Estudiant, Email=usuari1@gmail.com, Nom=A, Adreca=Plaça Universitat,
Num. prestecs normals=0, Num. prestecs llargs=0
```

Un cop creada la classe `Usuari`, procedim a definir dues classes filles, `Estudiant` i `Professor`, que implementaran els següents mètodes que defineixen les quantitats màximes de prestecs normals i llargs per a cadascú:

```
public int getMaxPrestecsNormals();
public int getMaxPrestecsLlargs();
```

La interfície `InUsuari` proporcionada al codi base ha de ser implementada per la classe `Usuari`.

Classe Prestec i Classes Filles

La classe `Prestec` s'utilitza per gestionar els prestecs. Per tal de vincular un exemplar amb un usuari, es recomana que la classe `Prestec` contingui com a atributs un objecte de cada tipus. També haurà de tenir un atribut de tipus `Date`, que permetrà emmagatzemar la data de creació de la comanda, i un altre atribut del mateix tipus per registrar la data límit de retorn del prestec. Per poder declarar objectes de la classe `Date`, serà necessari importar `java.util.Date`. Si es crea un objecte de la classe `Date` amb `new` sense paràmetres, l'objecte emmagatzema la data actual. Per acabar, la classe `Prestec` també tindrà un atribut de tipus `boolean` que es farà servir per saber si un prestec ha estat retornat.

La classe `Prestec` es definirà com a classe abstracta que incorporarà un constructor i els getters i setters necessaris.

També definirem el mètode `toString`, que ens permetrà obtenir la informació associada a un prestec d'una manera semblant a la següent:

```
Tipus=Llarg, Exemplar=El Quijote, Usuari=A, Data de creacio=Thu Mar 05
17:12:03 CET 2026, Data límit retorn=Thu Mar 05 17:14:23 CET 2026, Retor-
nat=false
```

Un cop definida la classe `Prestec`, procedirem a crear les classes filles `PrestecNormal` i `PrestecLlarg`, que implementaran el mètode per a indicar el tipus de préstec ("Normal" o "Llarg") i la durada del préstec en mil·lisegons:

```
public String tipusPrestec();
public long duradaPrestec();
```

El codi base incorpora la interfície `InPrestec`, que haurà de ser implementat per la classe `Prestec`.

Classe Llista i Classes Filles

L'aplicació que hem d'implementar ha de manejar col·leccions de tres tipus d'objectes: `Exemplar`, `Usuari` i `Prestec`.

Atès que les operacions que necessitem dur a terme són molt similars per a les tres col·leccions, la nostra intenció és implementar la majoria d'aquestes operacions en una classe genèrica, i a partir d'ella derivar classes per a les tres llistes que necessitem.

Les classes genèriques en Java constitueixen una eina molt valuosa per a reutilitzar programari. En concret, són classes similars a les classes convencionals, però que reben tipus com a paràmetre a l'hora d'instanciar-se. L'esquelet de la classe genèrica que desenvoluparem en aquesta pràctica ve donat com a material de la pràctica en el fitxer `Llista.java`, i es declara de la manera següent:

```
public class Llista<T> implements Serializable {
    protected ArrayList<T> llista;
    ...
}
```

En la declaració anterior, `T` constitueix un paràmetre de la classe que podrem canviar segons ens convingui. En l'exemple, l'ús d'aquest paràmetre incideix en el tipus d'elements que es gestionen en l'atribut `llista`, de tipus `ArrayList`.

A partir de la classe `Llista`, podem definir tres classes derivades: `LlistaExemplars`, `LlistaUsuaris` i `LlistaPrestecs`, segons el tipus d'objecte a manejar. Per exemple, la definició de `LlistaExemplars` té la forma:

```
public class LlistaExemplars extends Llista<Exemplar> implements
 Serializable ...
```

A més, la definició de classes derivades ens serà útil per redefinir mètodes de la classe `Llista`, ja que serà necessari gestionar les següents excepcions:

- En `LlistaExemplars` no es podran afegir dos exemplars amb el mateix identificador.
- En `LlistaUsuaris` no es podran afegir dos usuaris amb el mateix correu electrònic.

En aquestes classes s'han d'implementar els bucles fent servir Iterators.

Al codi base, es proporciona la interfície `InLlista`, que ha de ser implementada per la classe `Llista`

Classe Dades

La classe `Dades` és la classe principal del paquet del model, ja que, conté i gestiona totes les dades de l'aplicació.

Per establir amb claredat el que ha de fer aquesta classe, s'ha creat la interfície `InDades`, que proveeix una llista amb els mètodes a implementar.

Els mètodes proveïts per `Dades` seran utilitzats per la classe `Adaptador`. No hauria de ser necessari crear mètodes addicionals dins de `Dades` a més del constructor i els mètodes de `InDades`.

Addicionalment, des de la classe `Dades`, serà necessari llançar excepcions quan passin aquests esdeveniments:

- Afegir un objecte de tipus `PrestecLlarg` per a un exempler que no admet prestec llargs.
- Fer un prestec d'un exemplar no disponible.
- Fer un prestec a un usuari que té prestecs endarrerits.
- Fer un prestec a un usuari que excedeix el seu limit de prestecs normals o prestecs llargs.

Classe Adaptador

La classe `Adaptador`, com ja s'ha explicat anteriorment, serà utilitzada per intervenir entre la vista i les dades del model. Per això, la classe `Adaptador` farà servir una instància de la classe `Dades`, per ser la classe principal del nostre model.

Un exemple de servei proporcionat per la classe `Adaptador` seria convertir la llista d'objectes de classe `Exemplar` que retorna la classe `Dades` a una llista de `String`. Això permet que la vista funcioni sense tenir coneixement dels detalls d'implementació del model.

Tal com es veia en l'apartat 2, la nostra aplicació ha de ser capaç de guardar i carregar les dades del model a partir de fitxers (persistència de les dades). Es suggereix que aquesta tasca s'implementi mitjançant dos mètodes de la classe `Adaptador`, amb el prototip:

```
public void guardaDades(String camiDesti) throws BiblioException
;
public void carregaDades(String camiOrigen) throws
BiblioException;
```

Perquè les dades del model puguin carregar-se i guardar-se correctament, es recorda que les classes involucrades han d'implementar la interfície `Serializable`.

Si es necessita informació addicional sobre el tractament de la persistència de les dades en Java, es pot consultar el material de suport per a la pràctica anterior (Pràctica 2) disponible al Campus Virtual.

Classe BiblioUB

Defineix el menú d'opcions i la lògica de l'aplicació. Per a això, definirem una funció anomenada `gestioBiblioUB` amb la següent signatura:

```
public void gestioBiblioUB();
```

Aquesta funció internament utilitzarà la classe `Menu` que es proporciona com a material de la pràctica.

És molt important ressaltar que des de `BiblioUB` no es pot importar cap classe del paquet model. Només podem importar la classe `Adaptador`.

Classe IniciadorBiblioUB

Conté la classe principal de l'aplicació i està ubicada al paquet `prog2.vista`. Defineix un objecte de la classe `BiblioUB` i crida a mètode `gestioBiblioUB` esmentat anteriorment.

Classe BiblioException

S'ha de crear la classe `BiblioException` dins el paquet `prog2.vista`. Aquesta classe serà utilitzada per al maneig de les excepcions, tal com s'explica més avall.

6 Excepcions

Igual que en la pràctica anterior, la gestió dels errors es durarà a terme mitjançant excepcions. Un exemple d'operació que pot causar una excepció seria si intentem afegir un nou exemplar amb el mateix codi que un altre prèviament afegit, tal com s'esmentava en l'apartat anterior.

El primer pas per implementar el maneig d'excepcions és crear la classe `BiblioException`. Un cop creada (i tal com vam veure en la pràctica anterior), hem de fer servir `throw` des dels mètodes en què es pugui produir algun error. Farem servir la paraula clau `throws` al final de la capçalera del mètode en cas que vulguem delegar la captura d'una excepció fins al mètode on s'hagi de gestionar. El constructor de l'excepció ha de servir per introduir el missatge descrivint l'error. Finalment, utilitzarem un bloc `try-catch` per capturar l'excepció i informar l'usuari de l'error produït i controlar el flux d'execució.

Es recorda que es pot consultar informació addicional sobre el maneig d'excepcions en el material de suport de les pràctiques de l'assignatura disponible al Campus Virtual.

7 Classes de Test

Per comprovar el correcte funcionament del codi implementat és necessari córrer un conjunt de casos de prova (Unit Tests) que heu de definir en diverses classes de test corresponents a cadascuna de les classes del model i que hauran d'estar creades a la carpeta test. Un cas de prova (Unit Test Case) és una part de codi, que assegura que una altra part del codi (un mètode) funciona com s'esperava. Un cas de prova es caracteritza per una entrada coneguda i un resultat esperat.

JUnit és un marc de proves unitàries per al llenguatge de programació Java (https://junit.org/junit5/, https://www.jetbrains.com/help/idea/junit.html). Teniu un document de suport sobre JUnit al Campus Virtual.

8 Format del Lliurament

La Pràctica 3 es donarà per acabada quan s'acabin les dues parts que la componen i es corregirà llavors. No obstant això, cal fer el lliurament de la Pràctica 3 Part 1 junt amb una memòria que contindrà només dos diagrames.

Es recomana complir la data de lliurament de la tasca oberta en el campus virtual per poder començar el treball de la part 2 a continuació. El treball de la part 2 consistirà en el desenvolupament d'un altre projecte de IntelliJ que està descrit a un enunciat separat corresponent a la part 2 de la pràctica.

Pel lliurament de la Pràctica 3 Part 1 s'ha de generar un fitxer comprimit (ZIP) amb el nom dels membres de la parella: `Cognom1Nom1Cognom2Nom2_P31`, que contingui:

1. El projecte de IntelliJ: Tot el codi generat ha d'estar correctament comentat per poder executar el javadoc, generant automàticament la documentació en línia del codi.

2. Una memòria amb:

   - El diagrama de relacions entre les classes que has utilitzat a la pràctica 3 part 1. No cal incloure la llista d'atributs i mètodes.
   - El diagrama de seqüencia per mostrar la interacció entre objectes quan s'executa l'opció d'afegir un prestec.

NOTA IMPORTANT: de cara a les proves de pràctiques, el fitxer comprimit ha de ser enviat per tots els membres que participen en el lliurament.

9 Data Límit del Lliurament

Consulteu la data límit de la tasca oberta al Campus Virtual.